Project Status Report: BragBoard - Week 3 Completion

Project Name: BragBoard: Internal Employee Recognition Wall

Date: October 20, 2025

Status: All Week 3 tasks are complete.

Name-E.Saipavan

GitHub-saipavanesikela/Bragboard

1. Overview of Week 3 Tasks

This week's focus was on implementing the core functionality of the BragBoard application: allowing a logged-in user to create a shout-out and tag their colleagues. This is the first part of Milestone 2: Shout-Out Posting & Feed1.

The specific tasks for Week 3 were:

Create a shout-out form with recipient tagging2.

Store the shout-out in the database with tagged users3.

2. What We Have Done (Implementation Details)

We successfully implemented this feature by building the complete "Create Shout-Out" flow, from the frontend user interface to the backend database storage.

On the Backend (Storing the Data):

Database Models Created: We added the Shoutout and ShoutOutRecipient tables to our database models. This structure allows a single shout-out to be linked to multiple recipients, fulfilling the tagging requirement 4.

Schemas Defined: We created Pydantic schemas (ShoutoutCreate) to validate the incoming data, which includes a message (string) and a recipient_ids (list of numbers).

CRUD Logic Implemented: We wrote the create_shoutout function, which:

Creates the main Shoutout record.

Loops through the list of recipient IDs and creates a ShoutOutRecipient record for each one, linking them all to the new shout-out.

API Endpoints Built:

We created a new POST /api/v1/shoutouts/ endpoint that uses the get_current_user dependency. This ensures only a logged-in user can post, and it correctly identifies them as the sender.

We also created a new GET /api/v1/users/ endpoint to securely provide a list of all users to the frontend.

On the Frontend (The User Interface):

"Create Shout-Out" Modal: Instead of a separate page, we created a pop-up modal (CreateShoutoutModal.jsx). This is a better user experience as it doesn't navigate the user away from their feed.

Recipient Tagging: We connected the modal to the new /api/v1/users/ endpoint. The modal now fetches a list of all employees and displays them in a <select multiple> dropdown. This allows a user to select one or more colleagues to tag in their post.
Full End-to-End Connection: The "Post" button in the modal is now fully functional. It sends the message and the list of selected recipient IDs to the backend, where they are stored in the database.

3. Problems Faced and How We Solved Them

We encountered several common but critical errors during integration.

Problem: 404 Not Found error when the frontend tried to fetch data from /api/v1/users/me or /api/v1/shoutouts/.

Solution: This was a backend configuration error. The new router files (users.py, shoutouts.py) were created but had not been connected to the main application. We fixed this by adding api_router.include_router(...) for each new endpoint in the backend/app/api/v1/api.py file.

Problem: The backend crashed with multiple AttributeError messages (e.g., module 'app.schemas' has no attribute 'Shoutout').

Solution: This was a structural issue with Python's imports. The imports were too general (e.g., from app import schemas). We fixed this by refactoring the code to use specific, direct imports from the exact file (e.g., from app.schemas.shoutout import Shoutout, ShoutoutCreate), which resolved the errors.

Problem: The frontend was blank and the console showed SyntaxError: The requested module ... does not provide an export named ...

Solution: This was a simple but critical error. The apiService.js file was missing the export keyword for the new functions (like getShoutouts and getCurrentUserProfile). We fixed this by adding export const before each function, making them available to the components that needed them.

4. Conclusion & Next Steps

We have successfully completed all Week 3 tasks. The application now supports its primary feature: a logged-in user can create a shout-out and tag multiple recipients, and this data is correctly stored in the database.

The project is now perfectly set up for the Week 4 task: "Display all shout-outs on the feed"5.